SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1412 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

Duration :60 Mins

QUESTIONS: 3 **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I Bhavya(192524376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Bhavya S

5/8/24

CO2-AT:3

1. a) Construct CFG:

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

This grammar generates expression such as

(id+id)*id

id+(id*(id+id))

id*id+id

(id)

b) 1. Correct Operator Precedence (* > +).

Multiplication has higher precedence.

Eg: id+id*id

Parsing Order: id*id

then, id+(id*id)

Hence * > +

2. Proper Handling of Nested Expressions:

The rule

$F \rightarrow (E)$

Eg: (id+(id*id))

The parse repeatedly replies

$F \rightarrow (E)$

c) The grammar is unambiguous

Justification:

Every valid expression has only one parse tree.

Eg: id+id*id

can be interpreted as

id+(id*id)

because multiplication is defined inside T

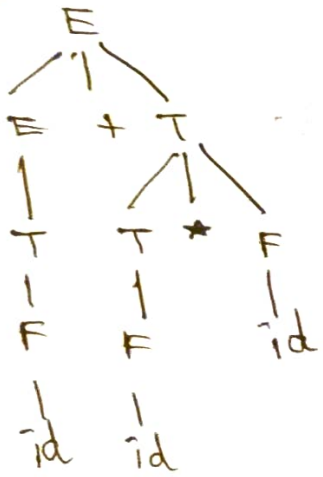
d) Leftmost Derivation:

Input:

id + id * id

E
⇒ E + T
⇒ T + T
⇒ F + T
⇒ id + T
⇒ id + T * F
⇒ id + F * F
⇒ id + id * F
⇒ id + id * id

e) Parse tree:



id + id * id

f) If an ambiguous grammar is used it can be modified into

E → E + E
E → E * E
E → (E)
E → id

E → E + T / T
T → T * F / F
F → (E) / id

→ This seperates +, * & id
→ Operator Precedence is maintained.
→ Left associativity is maintained
→ Ambiguity is removed.

2. Grammar

$S \rightarrow 0\ S1 \mid 01$

Input string

0011

a) **Shift - Reduce Parsing:**

→ Bottom - up parsing technique in which parser constructs the parse tree from the input symbols towards the start symbol.

Working:

1. **Shift** - Move the next input symbol onto the stack.
2. **Reduce** - Replace a sequence of symbols on the stack with the corresponding non-terminal according to the grammar.

b) $S \rightarrow 0\ S\ 1$

$S \rightarrow 01$

Input : 0011

$01 \rightarrow S \rightarrow 0S1 \rightarrow S$

c)

Stack	Input	Action
$	0011$	Start
$0	011$	Shift
$00	11$	Shift
$001	1$	Shift
$00S	1$	Reduce 01→S
$00S1	$	Shift
$S	$	Reduce 0S1→S
$S	$	Accept

d) Rightmost Derivation in Reverse:

S
$\Rightarrow 0S1$
$\Rightarrow 00S1$

Reverse (Bottom - Up)

$00\underline{11}$
$\Rightarrow 0\underline{S1}$
$\Rightarrow S$

e) The grammar is Unambiguous

Reason:

Every valid string has exactly 1 derivation & 1 parse tree.

Eg:

0011

can only be derived as

S
$\Rightarrow 0S1$
$\Rightarrow 0011$

f) During shift - reduce parsing, the parser continuously checks whether the symbol on the stack match any grammar production

Input:

0101

→ No valid handle is present
No reduction is possible
The remaining input cannot be parsed according to the grammar.

∴ The Parser reports a syntax error & rejects the string.

Reason:

0101 doesn't follow the recursive structure generated by $S \rightarrow 0S1 | 01$.